

EE2213 Introduction to Artificial Intelligence

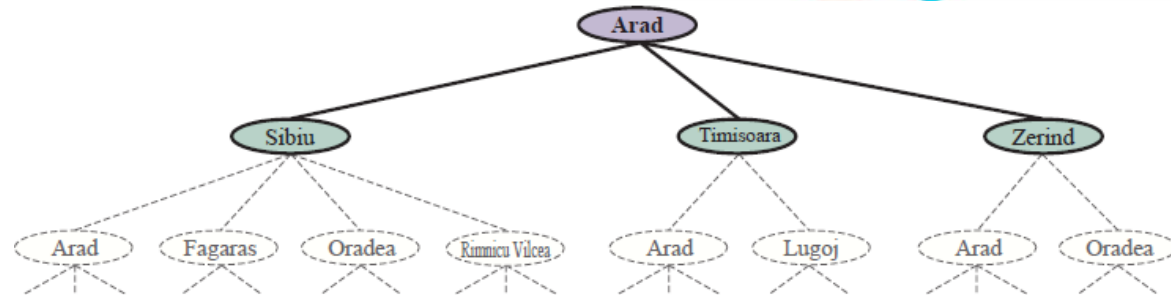
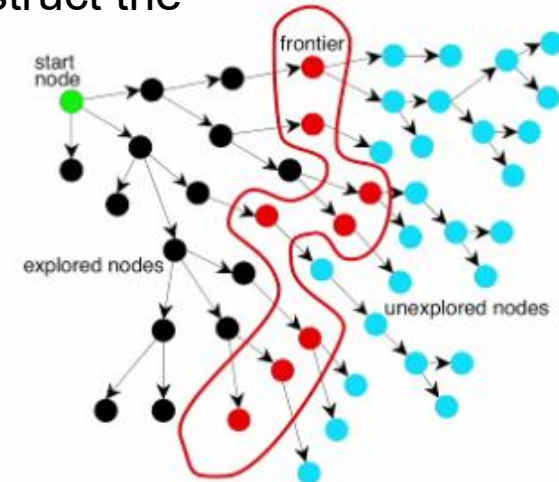
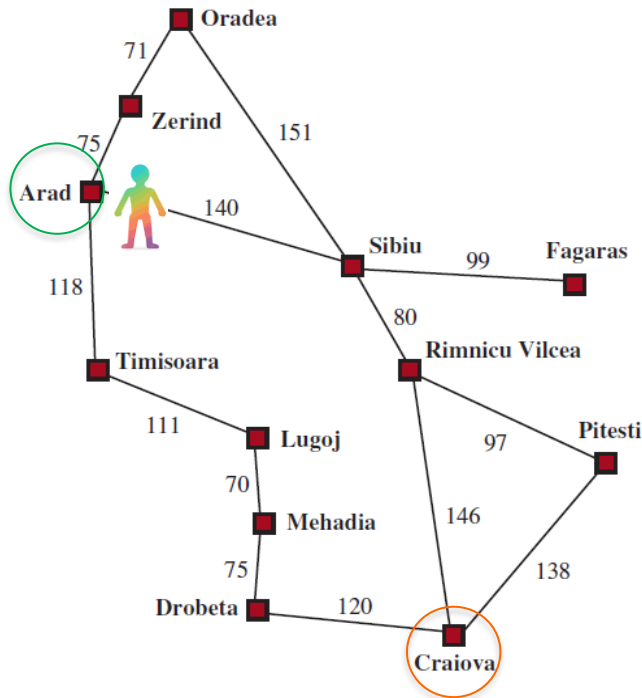
Lecture 3

Dr. Shaojing Fan
fanshaojing@nus.edu.sg

Recap: Frontier---Tracks the boundary between explored and unexplored

Frontier

- Queue (or other structure) of nodes **waiting to be expanded**
- **Stores state and path information**, needed to reconstruct the solution once the goal is reached



Reference: Stuart Russell & Peter Norvig, *Artificial Intelligence: A Modern Approach* (4th ed.) P90.

Reference: <https://www.youtube.com/watch?v=xoqcWXTXcl>

Recap: Graph search vs Tree search

Tree search

```
function TREE-SEARCH(problem, strategy) return a solution, or failure
  Initialize frontier to the initial state of the problem
  loop do
    if the frontier is empty then return failure
    choose a node from the frontier for expansion according to strategy,
    and remove it from frontier
    if the node contains goal state then return the corresponding solution
    else expand the node, and add the resulting child nodes to the frontier
  end
```

Graph search

```
function GRAPH-SEARCH(problem, strategy) return a solution, or failure
  Initialize frontier to the initial state of the problem
  Initialize the explored set to be empty
  loop do
    if the frontier is empty then return failure
    choose a node from the frontier for expansion according to strategy,
    and remove it from frontier
    if the node contains goal state then return the corresponding solution
    add the node to the explored set
    expand the node, and add the resulting nodes to the frontier
    only if not in the frontier or explored set
  end
```

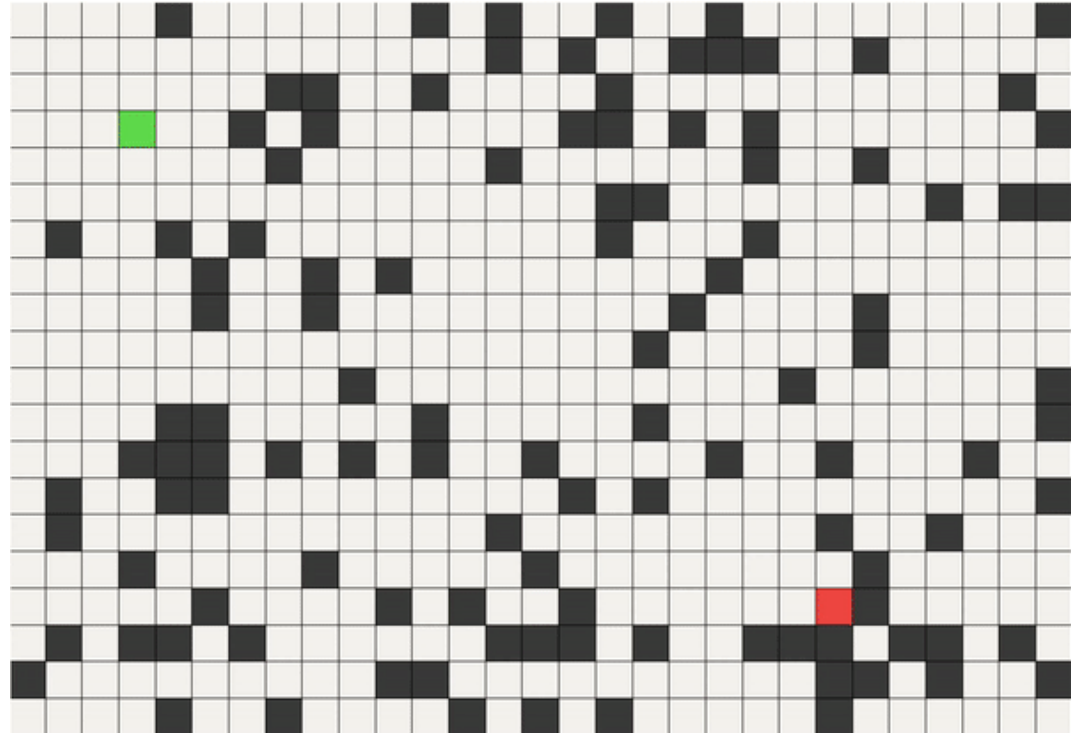
Reference: <https://www.youtube.com/watch?v=Y1VywhuRFIs>

Summary: BFS vs DFS

Feature	BFS (Breadth-First Search)	DFS (Depth-First Search)
Search style	Explores all nearby nodes first (level by level)	Goes as deep as possible before backtracking
Data structure used	Queue (FIFO - First in, first out)	Stack (LIFO - Last in, first out)
Will it always find a solution if one exists? (Completeness)	Yes, if a solution exists	Not guaranteed – may get stuck in deep or infinite paths
Will it find the shortest solution? (Optimality)	Yes, if all steps cost the same	No, may find a longer or less efficient path
Time needed	Slower if the solution is deep	Can be faster if the solution is deep
Memory needed	High – remembers all paths at one level	Low – tracks only one path at a time
Best for	Finding shortest paths (e.g., in maps) Small search space	Exploring puzzles or scenarios with deep solutions Large or deep search spaces
When to use	Need shortest route/minimum hops Equal step cost Enough memory	Memory is limited When solution may be far/deep Don't need shortest path

Depth-first search (DFS) in maze

- Expand the deepest unexplored node in the frontier first.
- *Explore one path as far as it can go before backtracking.*
- All the successors of a node are expanded before moving to other siblings.

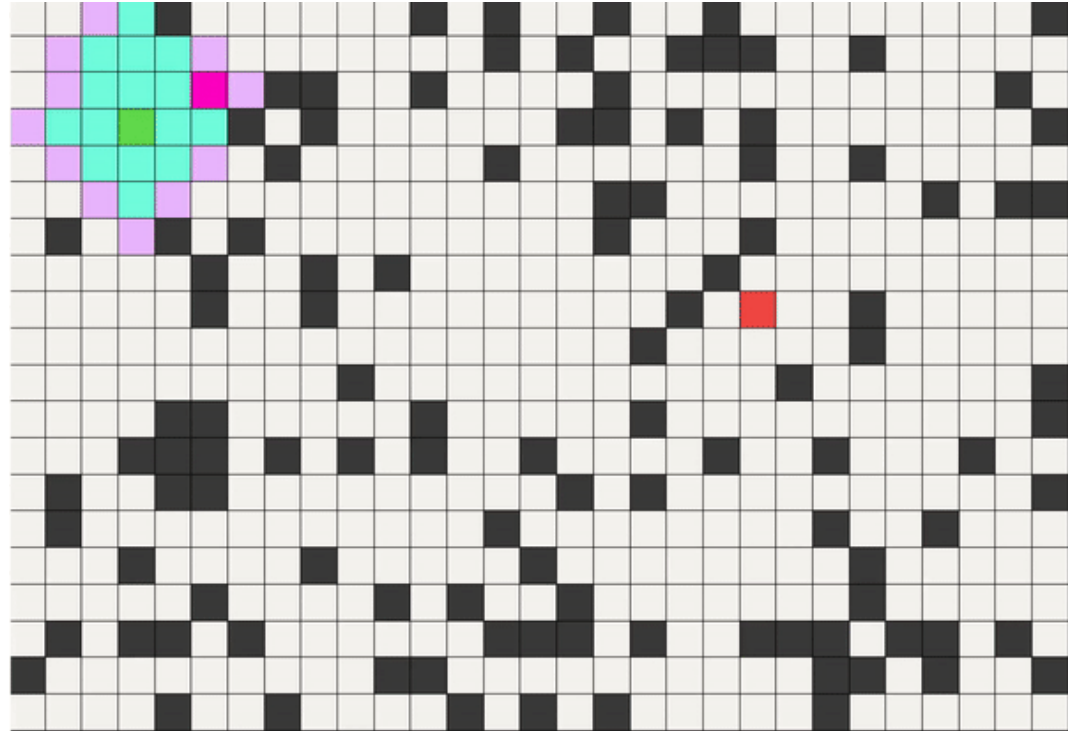


Source: <https://medium.com/codex/python-project-idea-graph-traversal-and-pathfinding-algorithm-visualisations-99595c414293>

Breadth-first search (BFS) in maze

- Explore nodes in “layers” (level by level)
- Can find everything findable from a given starting node
- Can compute *minimum hops between two nodes* (i.e., *shortest path*, if all steps cost the same)

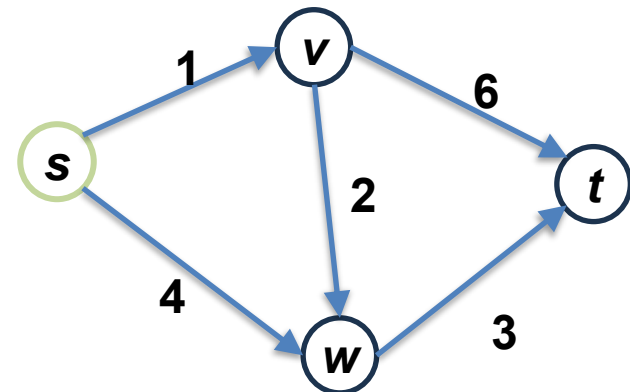
What if step costs are different?



Source: <https://medium.com/codex/python-project-idea-graph-traversal-and-pathfinding-algorithm-visualisations-99595c414293>

Quick question 1

Can we use BFS to find the shortest path from node s to node t ? The numbers on each edge represent the path distance of that edge.



The route with the fewest transfers isn't necessarily the fastest!



Limitation of BFS

BFS is effective for finding the shortest paths **only** in graphs where all the edges have the **same (unit)** length.

Uniform-Cost Search (UCS, also known as Dijkstra's algorithm) extends this by handling graphs with **any non-negative** edge length.

**Uniform-cost search (UCS) is the name used in the AI community, while the theoretical computer science community refers to it as Dijkstra's algorithm.*

Agenda

Uniform $\neq$ Uninformed

- Uniform-cost search (Dijkstra's algorithm)
- Informed search (greedy best-first, A*)

The **uniformity is in the way the algorithm fairly (uniformly) evaluates and expands paths **based on cost**, rather than by depth or number of steps.*

Application: Map navigation problems

BFS & DFS

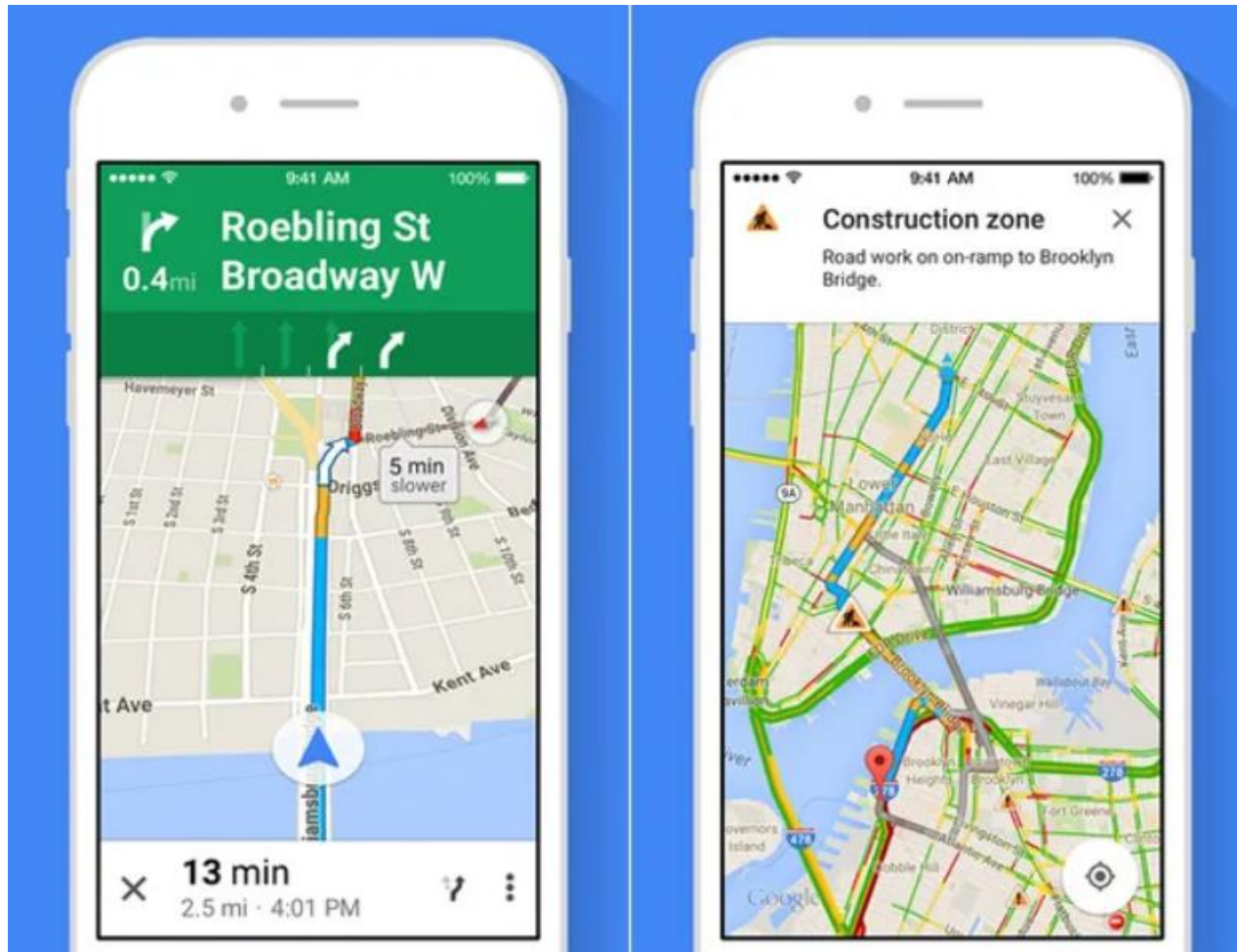
assume **step-cost**
are equal.

*This is only true
for some
problems.*

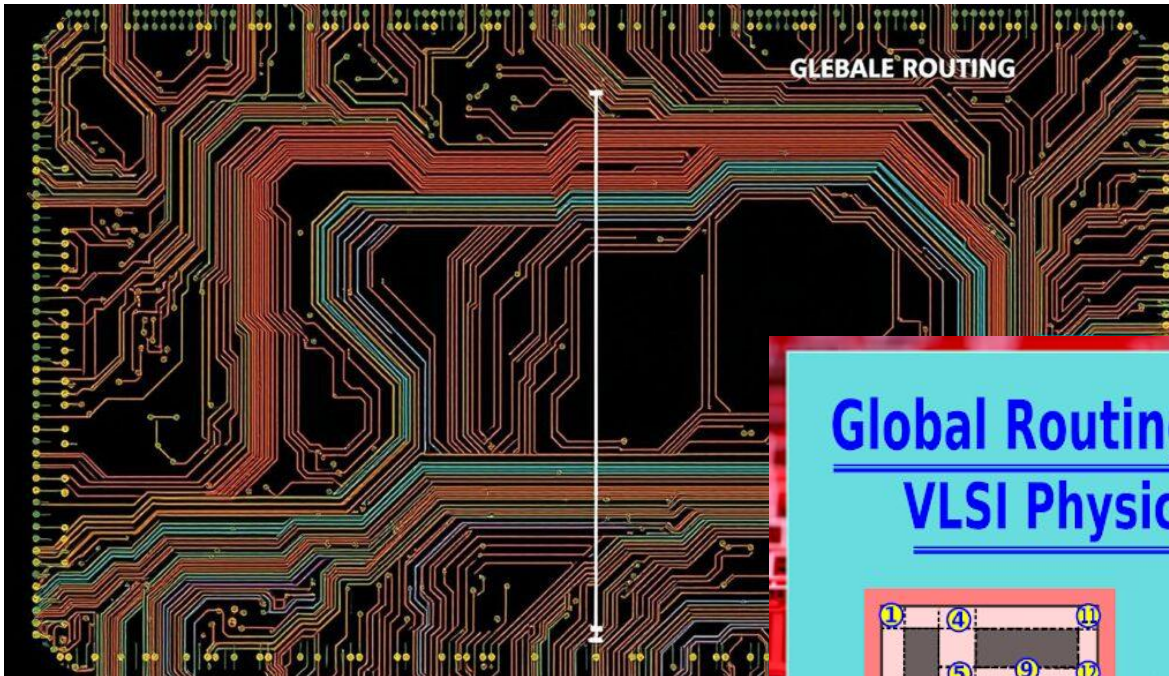
If **step costs**
vary, **Uniform-
Cost Search**
(**Dijkstra's
algorithm**) should
be used instead.



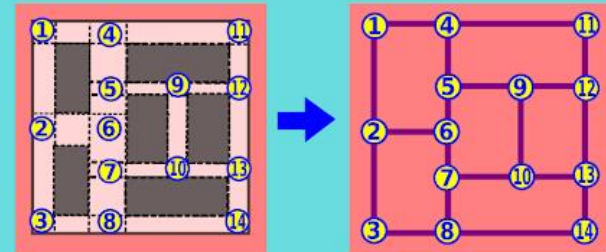
Application: Map navigation



Application: VLSI circuit design



Global Routing VLSI Physical Design



Dijkstra's algorithm & BFS

- Dijkstra's algorithm extends BFS for varying edge lengths
- Have a shared principle
- Similarity with equal edge lengths

Dijkstra's algorithm: Overview

Notation:

$X = \{ \}$

[explored set: nodes processed so far]

$Q = \{ \}$

[frontier: nodes waiting to be expanded]

$dist[v]$

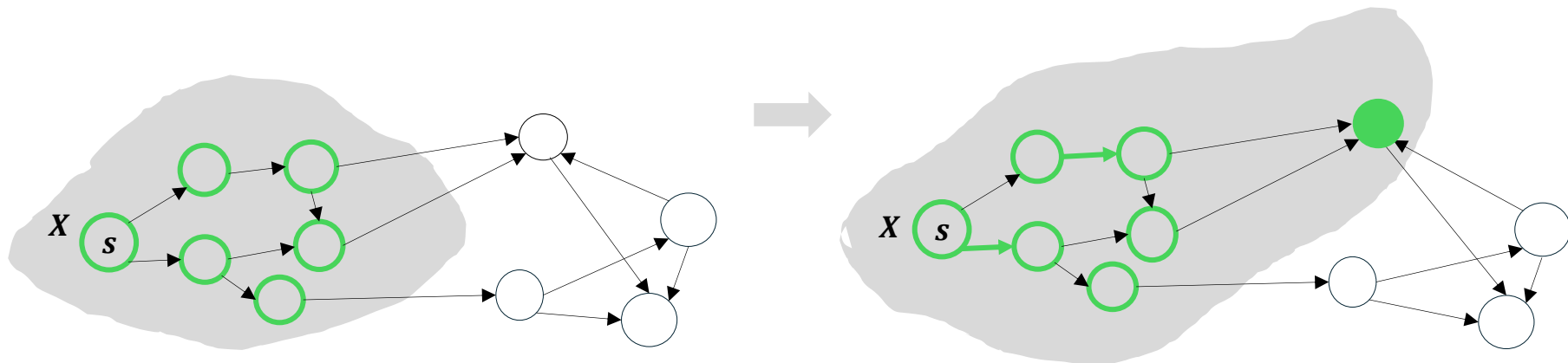
[shortest path distances from the start s to v]

$e(v, w)$

[edge from v to w]

$l(v, w)$

[length/cost of $e(v, w)$]



Dijkstra's algorithm: Overview

Initialize:

$$Q = \{s\}$$

[start from initial state s]

$$dist[s] = 0$$

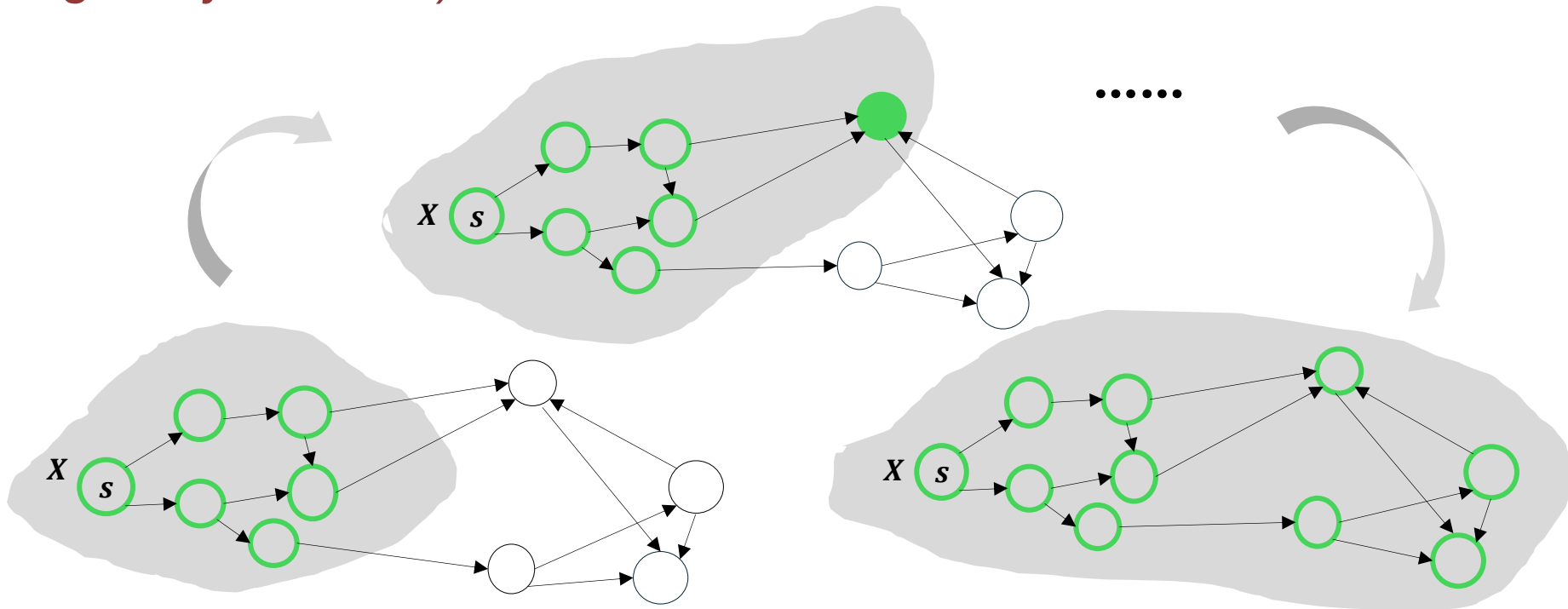
[computed shortest path distances]

Dijkstra's algorithm: Overview

Loop:

Among all $e(v, w) \in E$, with $v \in X, w \notin X$,

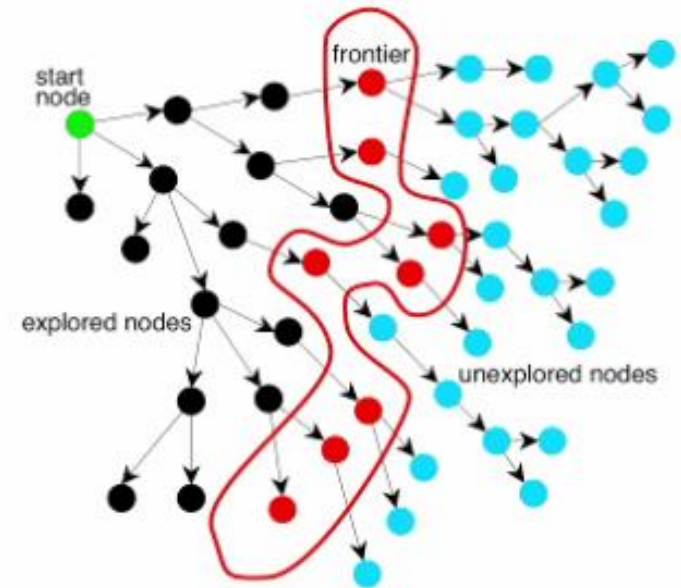
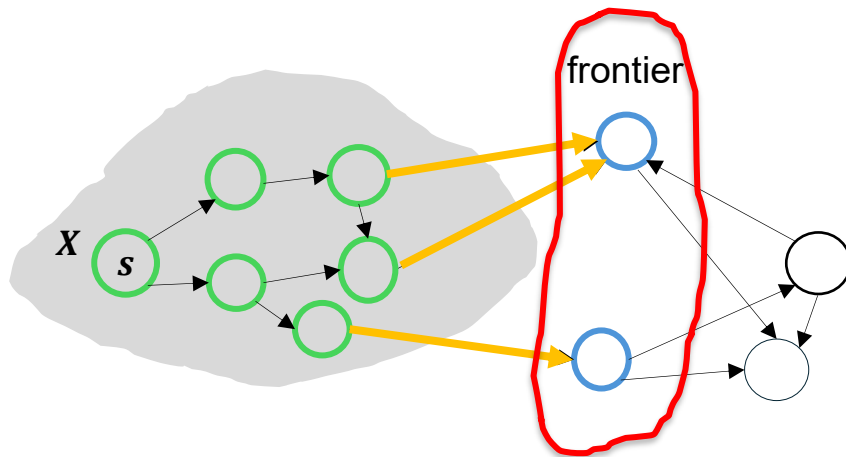
pick the one that minimizes $dist[v] + l(v, w)$ (*Dijkstra's greedy criterion*)



Dijkstra's algorithm: Overview

Loop:

Among all $e(v, w) \in E$, with $v \in X, w \notin X$,
pick the one that minimizes $dist[v] + l(v, w)$ (*Dijkstra's greedy criterion*)

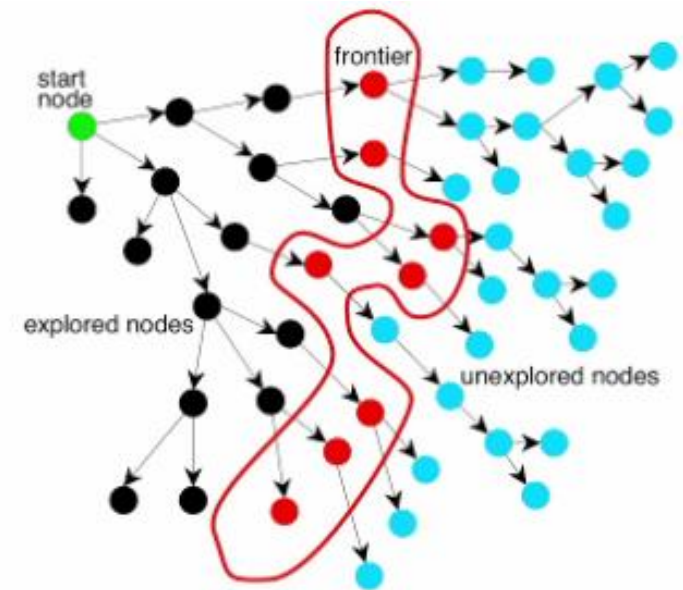
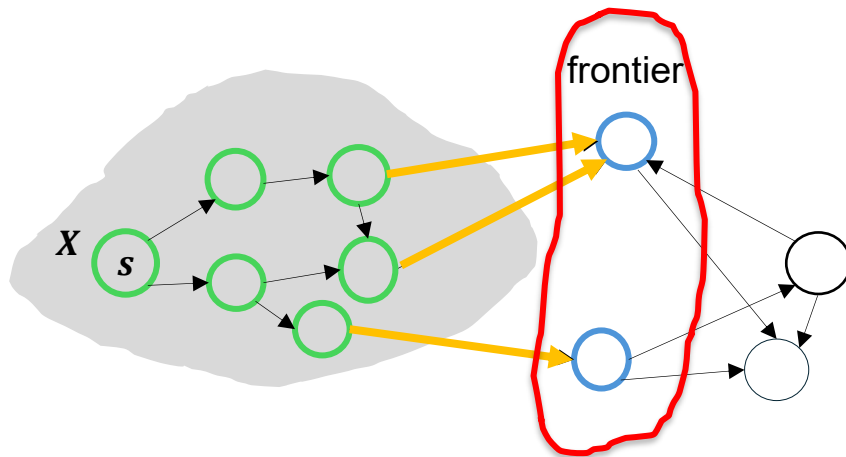


Dijkstra's algorithm: Overview

Loop:

Among all $e(v, w) \in E$, with $v \in X, w \notin X$,
pick the one that minimizes $dist[v] + l(v, w)$ (*Dijkstra's greedy criterion*)

From the frontier, choose the node closest to the source.

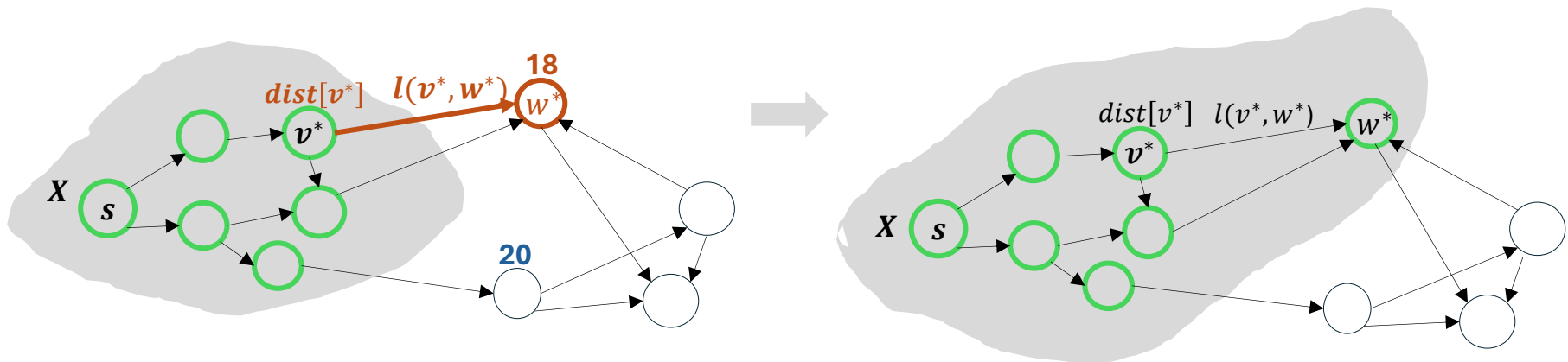


Dijkstra's algorithm: Overview

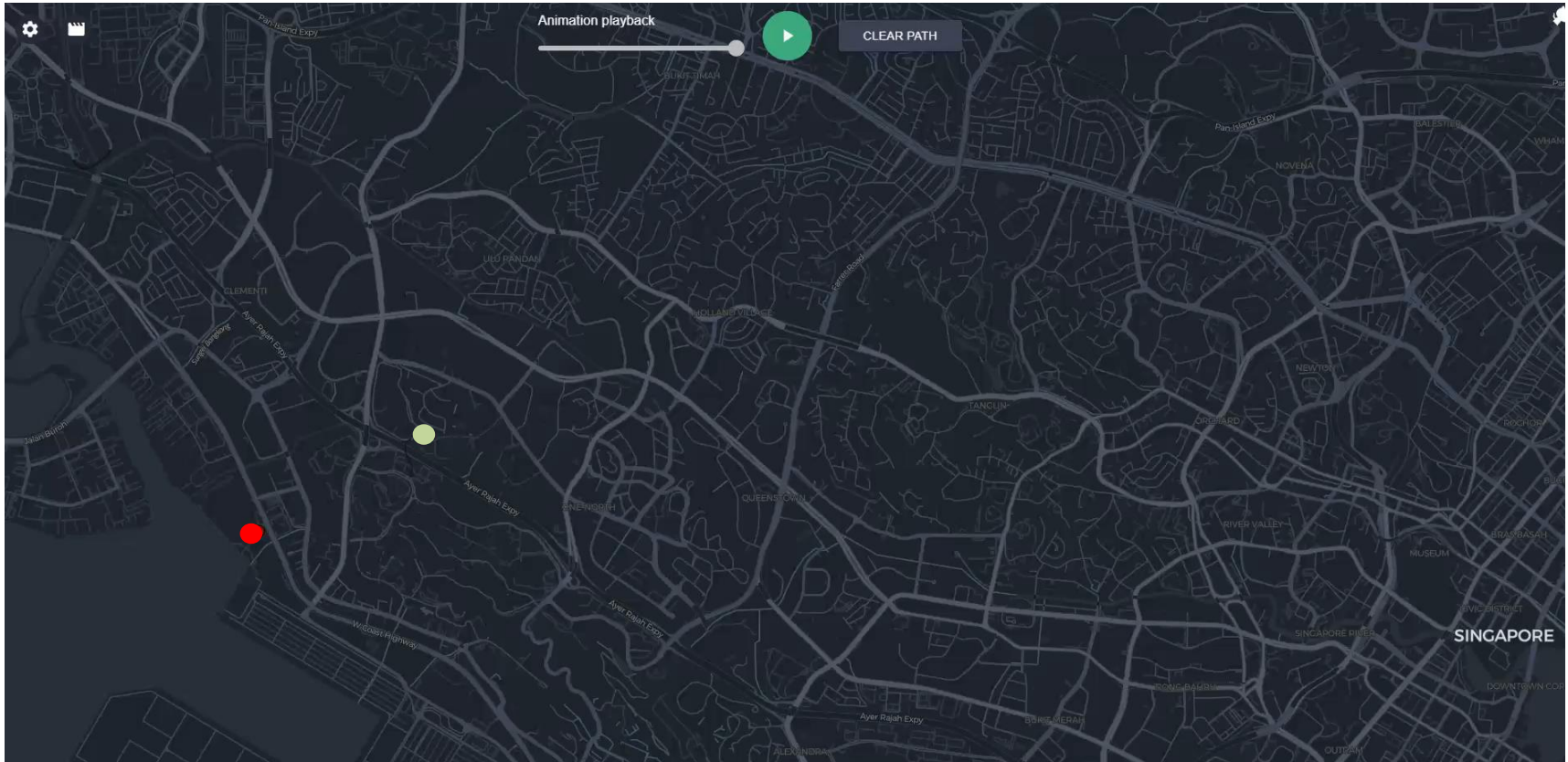
Loop:

Among all $e(v, w) \in E$, with $v \in X, w \notin X$,

pick the one that minimizes $dist[v] + l(v, w)$ (*Dijkstra's greedy criterion*), add w^* to X , $dist[w^*] = dist[v^*] + l(v^*, w^*)$



Dijkstra's algorithm: Visualisation

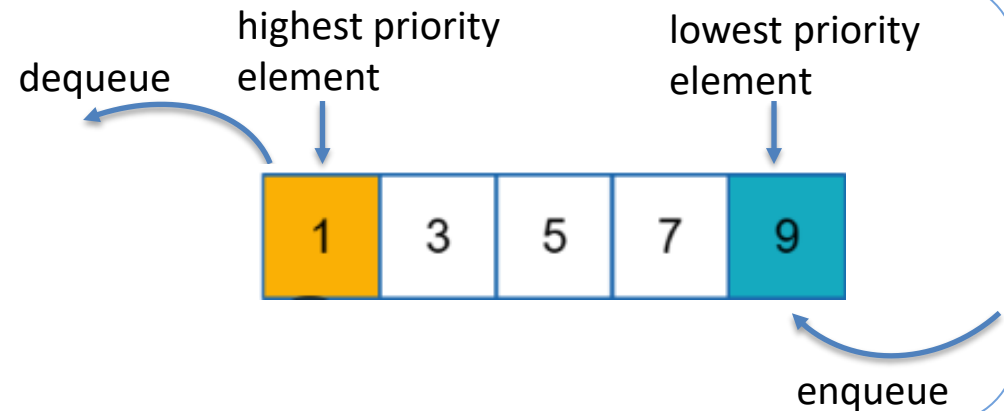
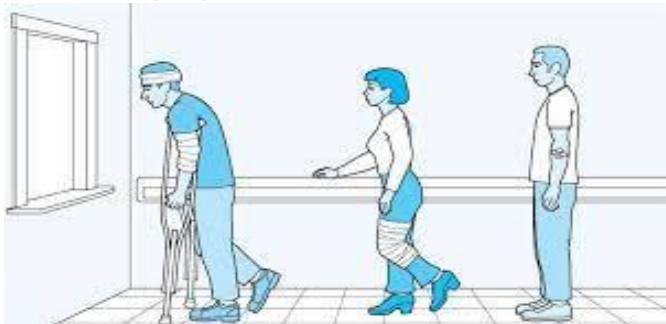


Courtesy: Pathfinding visualiser, Honzaap, <https://honzaap.github.io/Pathfinding/>

Dijkstra's algorithm: Frontier as a priority queue

- Dijkstra's algorithm always expands the frontier node with the *smallest distance from the source (initial state)*.
- In practice, the **frontier** is managed as a **priority queue**, so nodes in the frontier are expanded in order of their *current shortest distance from the source*.

Priority queue:



Reference: Stuart Russell & Peter Norvig, *Artificial Intelligence: A Modern Approach* (4th ed.) P94 – P95.

Dijkstra's algorithm (uniform-cost search): Pseudocode

```
function UNIFORM-COST-SEARCH(problem) returns a solution, or failure  
  node  $\leftarrow$  a node with STATE = problem.INITIAL-STATE, PATH-COST = 0  
  frontier  $\leftarrow$  a priority queue ordered by PATH-COST, with node as the only element  
  explored  $\leftarrow$  an empty set  
  loop do  
    if EMPTY?(frontier) then return failure  
    node  $\leftarrow$  POP(frontier) /* chooses the lowest-cost node in frontier */  
    if problem.GOAL-TEST(node.STATE) then return SOLUTION(node)  
    add node.STATE to explored  
    for each action in problem.ACTIONS(node.STATE) do  
      child  $\leftarrow$  CHILD-NODE(problem, node, action)  
      if child.STATE is not in explored or frontier then  
        frontier  $\leftarrow$  INSERT(child, frontier)  
      else if child.STATE is in frontier with higher PATH-COST then  
        replace that frontier node with child
```

Reference: Stuart Russell & Peter Norvig, *Artificial Intelligence: A Modern Approach* (3rd ed.) P84.

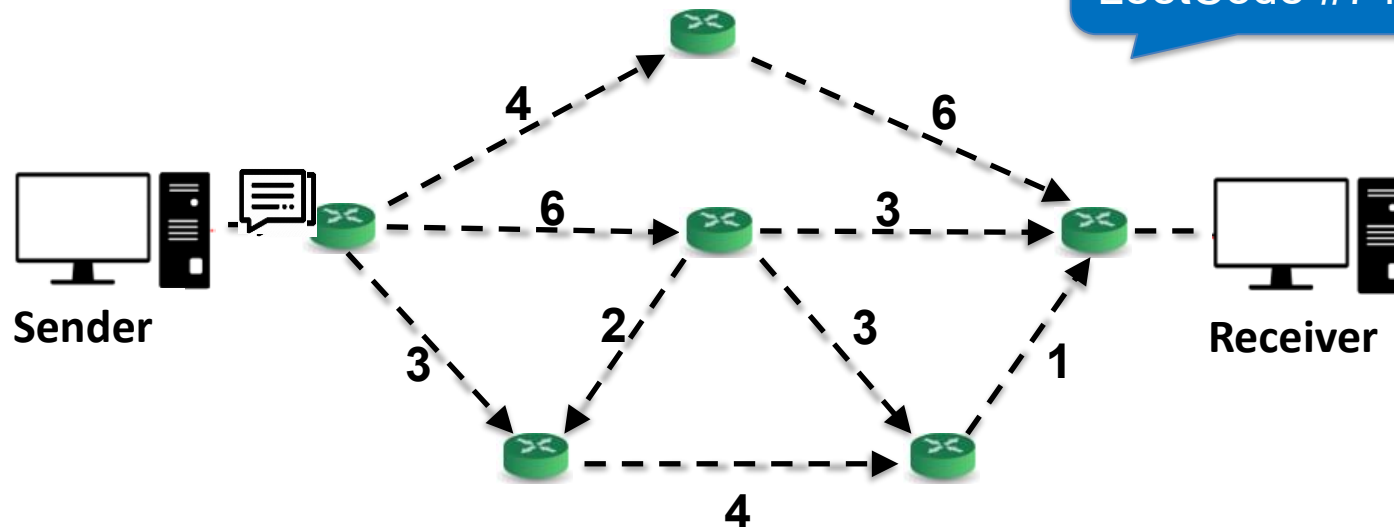
Dijkstra's algorithm (uniform-cost search): Pseudocode (v2)

- Start with a **frontier (priority queue)** containing the initial state (source), with path cost = 0.
- Start with an **empty explored set**.
- **Repeat**:
 - If the frontier is empty, then no solution.
 - Remove the frontier node with the **lowest path cost from the source**.
 - If this node is the goal, return the solution.
 - Add the node to the explored set.
 - Expand the node:
 - For each child, compute its **total path cost from the source**.
 - If the child is not in the frontier or explored set, **add it to the frontier** with its cost.
 - If the child is already in the frontier with a **higher cost**, update it with the **lower cost**.

If the new path is better than the old path, then discard the old one.

Dijkstra's algorithm in action: Shortest network delay

Adapted from
LeetCode #743

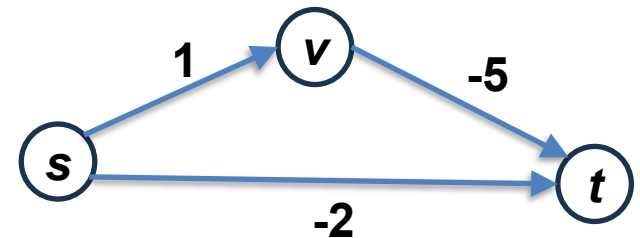


Refer to: [Lecture3_Dijkstra_Example.pdf](#)

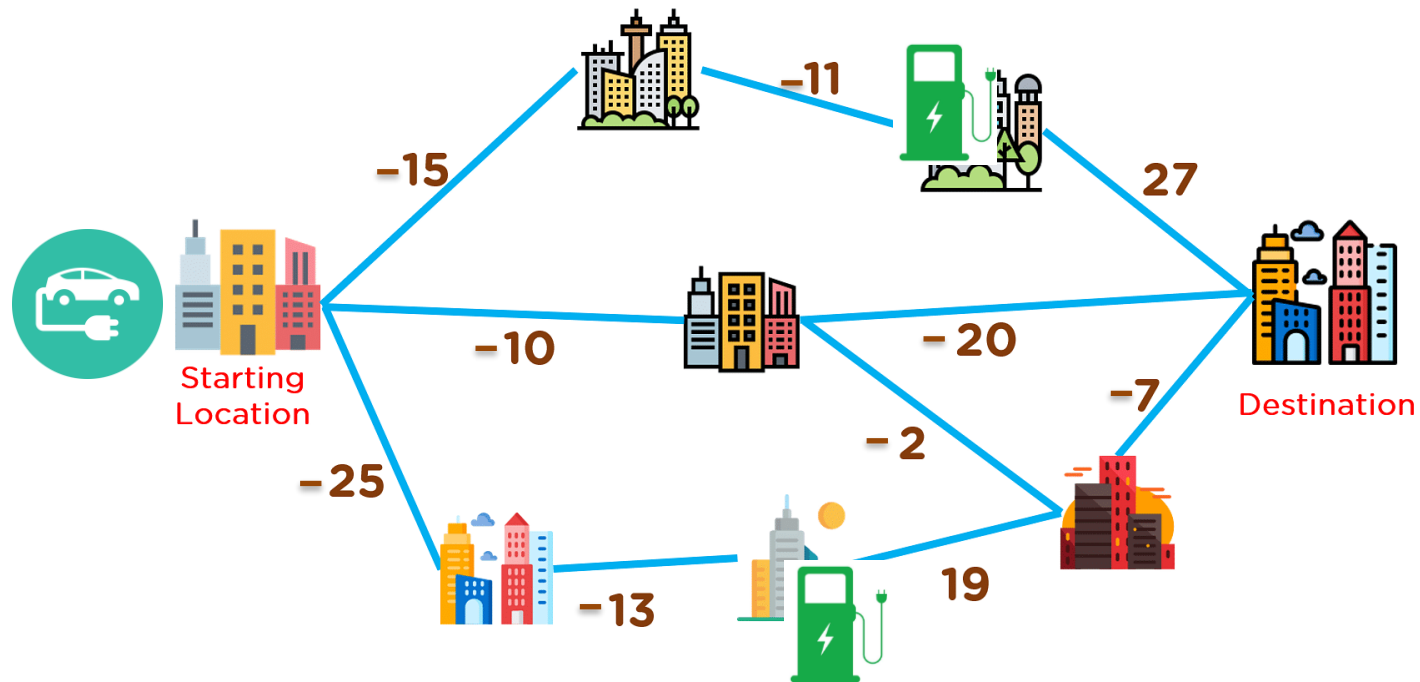
Quick question 2

What is the computed shortest distance from s to t , when running Dijkstra's algorithm on this graph? Assuming it treats the graph normally despite negative edges.

- (a) 1
- (b) -4
- (c) -2
- (d) ∞



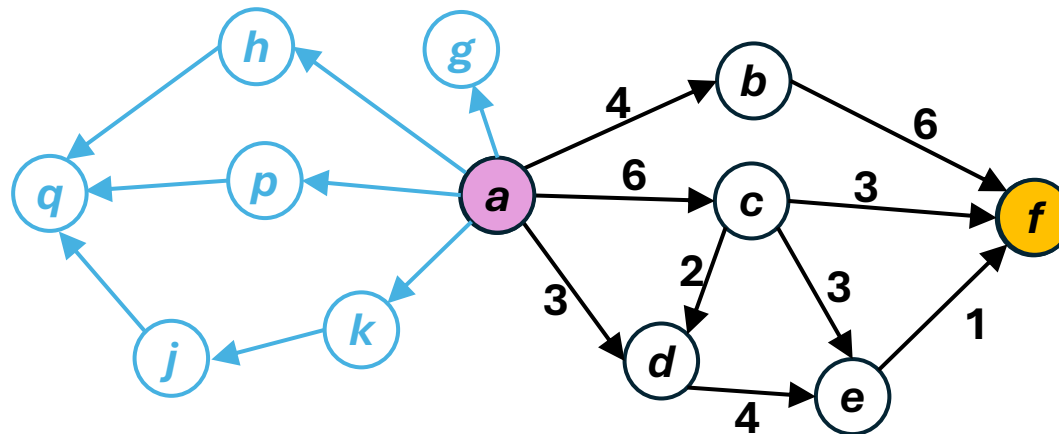
Negative edge cost: A real-world example



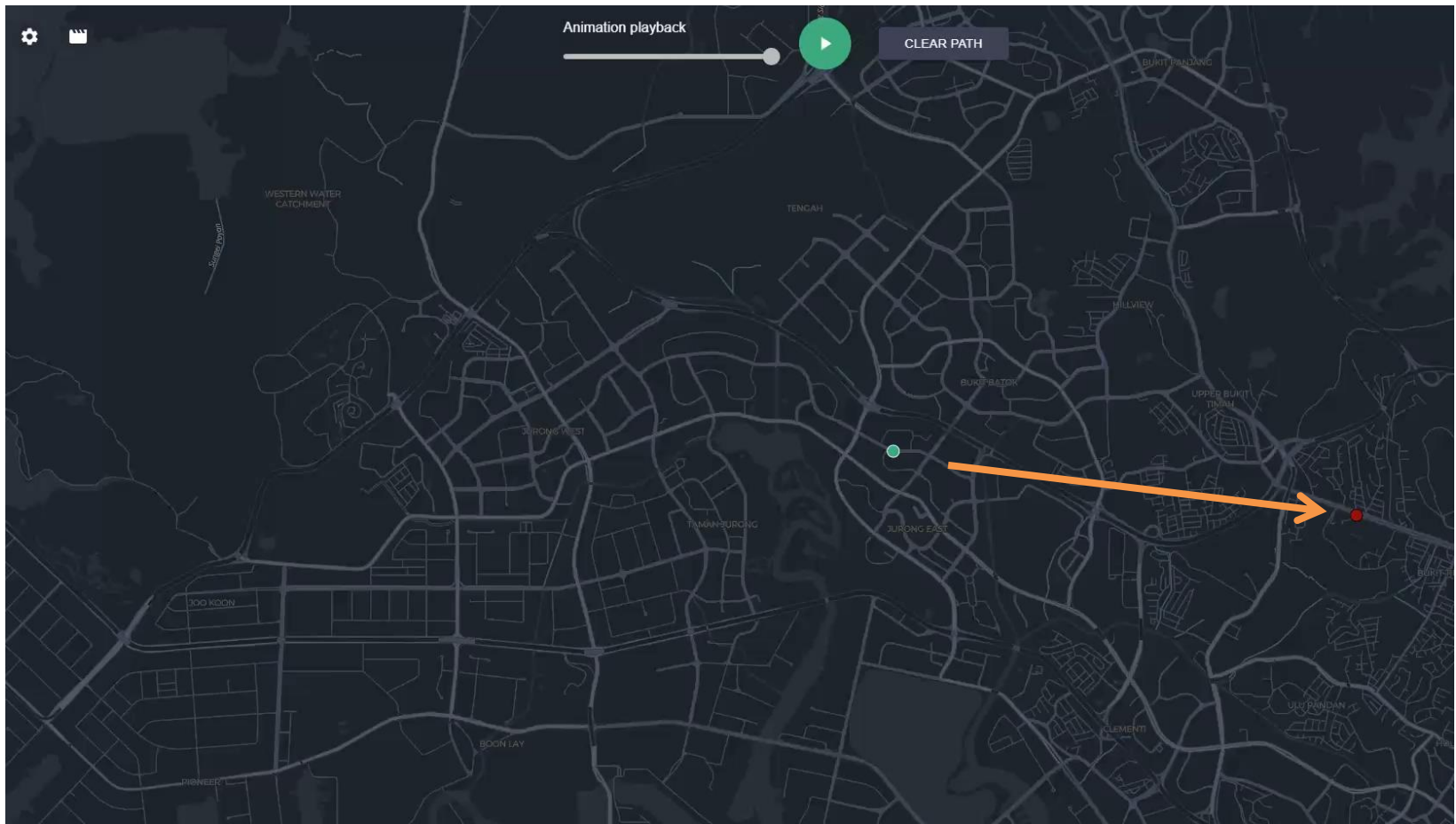
BFS vs DFS vs Dijkstra's algorithm

Feature	BFS (Breadth-First Search)	DFS (Depth-First Search)	Dijkstra's (Uniform-cost search)
Search style	Explores all nearby nodes first (level by level)	Goes as deep as possible before backtracking	Expands the node with the <i>lowest total cost</i> from the source (initial state)
Data structure used	Queue (FIFO - First in, first out)	Stack (LIFO - Last in, first out)	Priority queue (based on path cost)
Will it always find a solution if one exists? (Completeness)	Yes, if a solution exists	Not guaranteed – may get stuck in deep or infinite paths	Yes
Will it find the shortest solution? (Optimality)	Yes, if all steps cost the same	No, may find a longer or less efficient path	Yes, even when edge costs differ
Can it handle different edge costs?	No – assumes all edges have equal cost	No – ignores cost	Yes – works with any non-negative edge cost
Time efficiency	Slower if the solution is deep	Can be faster if the solution is deep	Slower in large graphs with varying costs
Memory usage	High – remembers all paths at one level	Low – tracks only one path at a time	High – must track cumulative costs for all paths
Best for	Finding shortest paths (e.g., in maps) Small search space	Exploring puzzles or scenarios with deep solutions Large or deep search spaces	Finding the cheapest/least-cost path in weighted graphs (e.g., shortest time, lowest expense)
When to use	Need shortest route Equal step cost Enough memory	Memory is limited When solution may be far/deep Don't need shortest path	Need shortest route with different edge costs; can afford higher memory usage

If the starting node a has neighbors in the opposite direction of the target vertex f , will Dijkstra's algorithm explore them? If so, how can we improve its efficiency?



Quick question 3 (example)



Experiment at: <https://honzaap.github.io/Pathfinding/>

Agenda

- Uniform-cost search (Dijkstra's algorithm)
- Informed search (greedy best-first, A^*)

Uninformed search & informed search



Uninformed search (BFS, DFS, Dijkstra's)

- Has no additional knowledge beyond what is given in the problem definition.
- Generate successor states and check whether each is a goal.
- As a result, the search proceeds blindly—systematically exploring the space until the goal is found.
- Search methods are distinguished by the order in which the nodes are expanded.

Informed search (greedy best-first, A*)

- Uses additional knowledge (heuristic) to estimate how close a state is to the goal.
- The heuristic guides the search, often reducing the number of nodes explored and improving efficiency.

Greedy best-first search

An informed search algorithm that expands the node that *appears to be closest to the goal*, as estimated by a heuristic function $h(n)$

$h(n)$: Estimated cost of the cheapest path from the current node n to a goal. (for goal node: $h(n) = 0$)

Heuristic function

Heuristic knowledge is useful, but NOT always accurate.

Heuristic algorithms use heuristic knowledge to guide the search process.

A **well-designed heuristic** can significantly reduce the number of nodes explored, making the search more efficient.

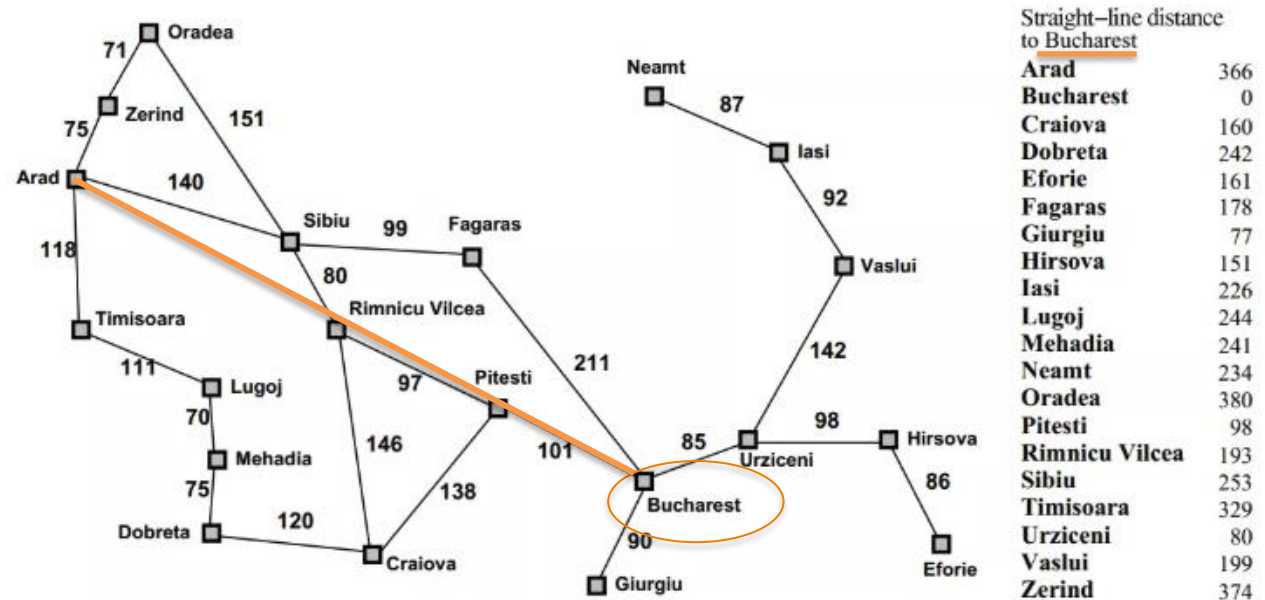
Archimedes (c. 287-212 BC)



Example: Heuristic in path planning

In map navigation, a common heuristic is the **straight-line (Euclidean) distance** to the goal.

This provides a reasonable estimate of how far a place is from the destination.



For two points $p = (x_1, y_1)$ and $q = (x_2, y_2)$, the Euclidean distance (also known as L2 distance) is:

$$d_{Euclidean}(p, q) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

$h(n)$

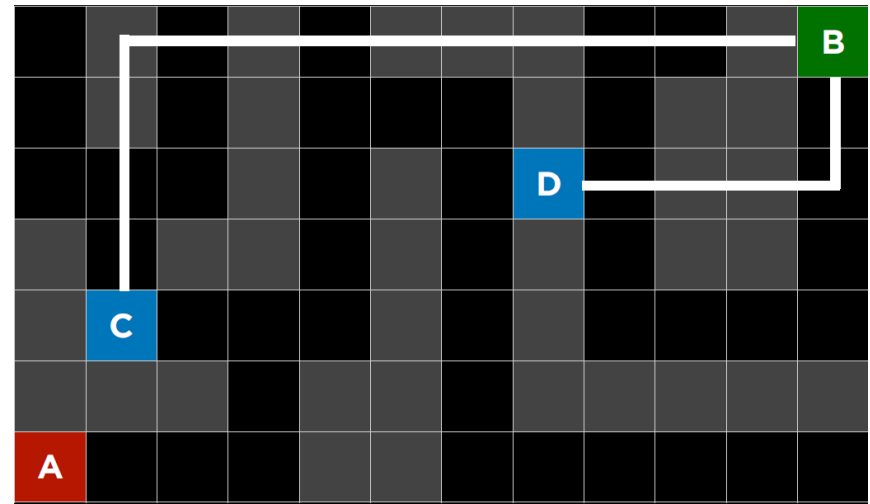
The heuristic is NOT always accurate, as actual travel paths are rarely straight due to roads, obstacles, and terrain.

Example: Heuristic in grid maze

Heuristic used: **Manhattan distance** (also known as **L1 distance**), which sums horizontal and vertical steps to the goal.

For two points $p = (x_1, y_1)$ and $q = (x_2, y_2)$, the Manhattan distance is:

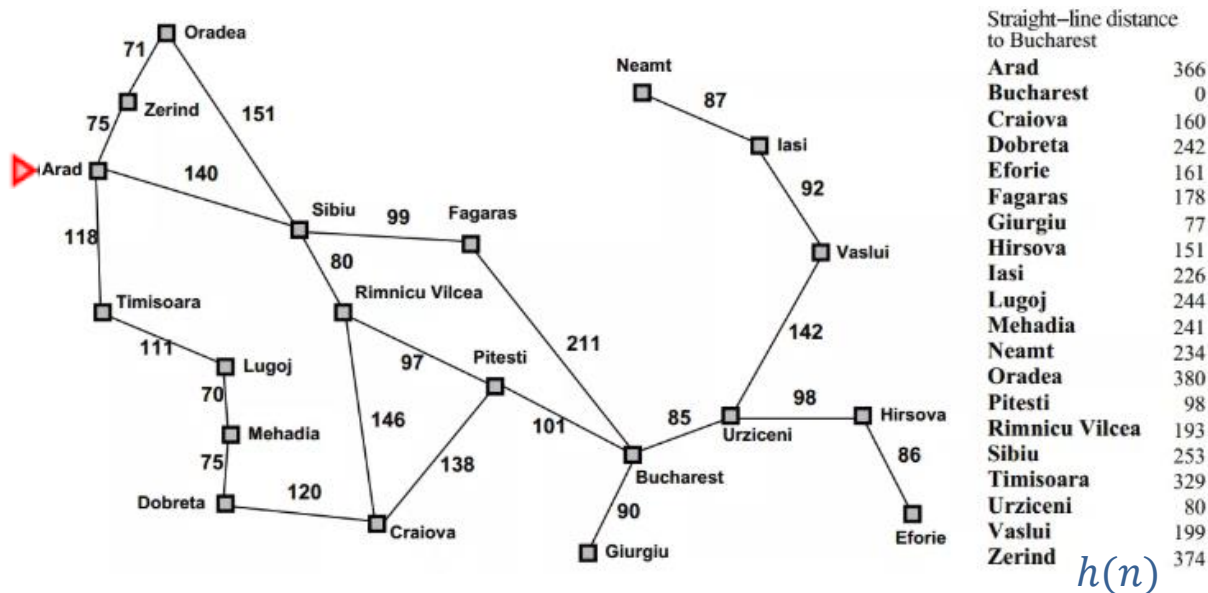
$$d_{\text{Manhattan}}(p, q) = |x_1 - x_2| + |y_1 - y_2|$$



11		9		7				3	2		B
12		10		8	7	6		4			1
13	12	11		9		7	6	5			2
	13			10		8		6			3
	14	13	12	11		9		7	6	5	4
			13			10					
A	16	15	14			11	10	9	8	7	6

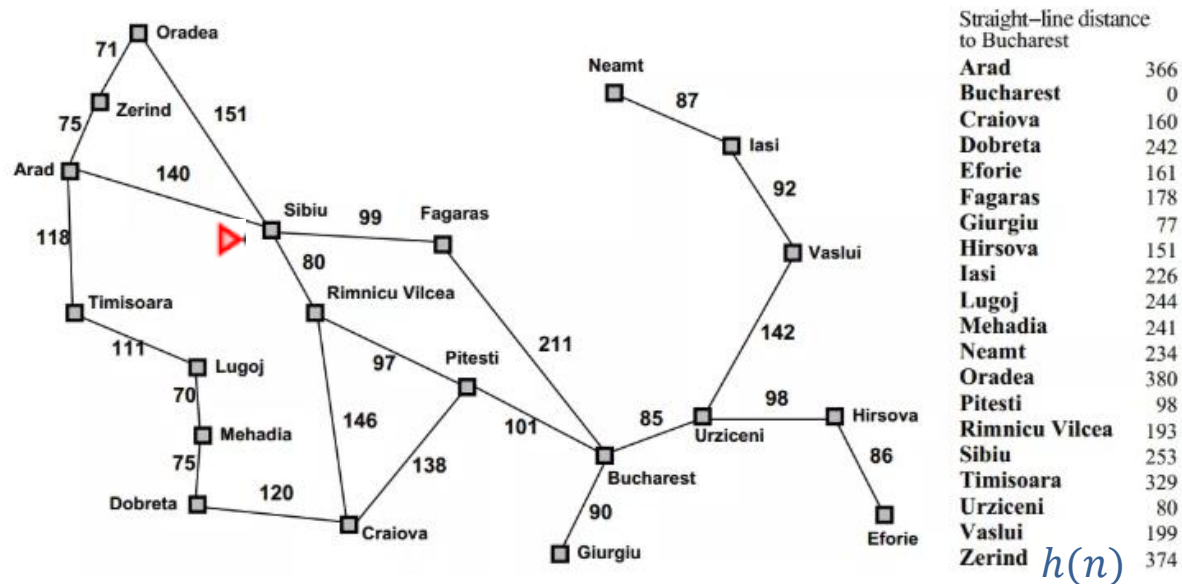
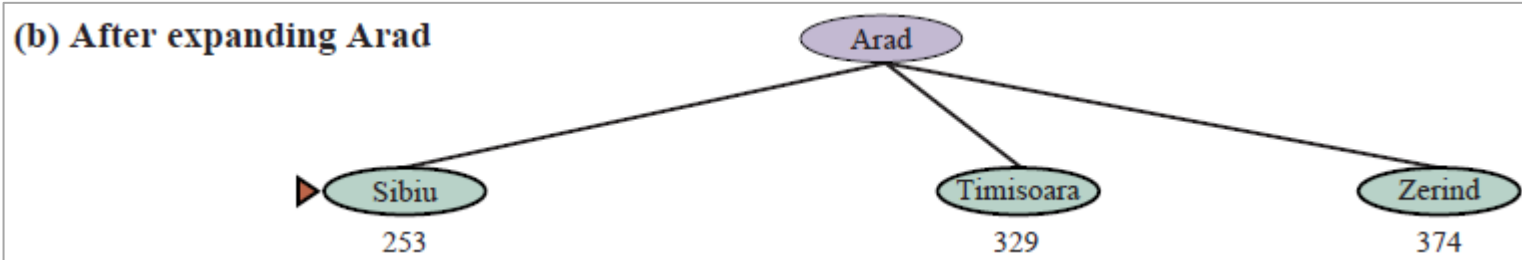
Path planning with greedy best-first

(a) The initial state



Reference: Stuart Russell & Peter Norvig, *Artificial Intelligence: A Modern Approach* (4th ed.) P103 – P104

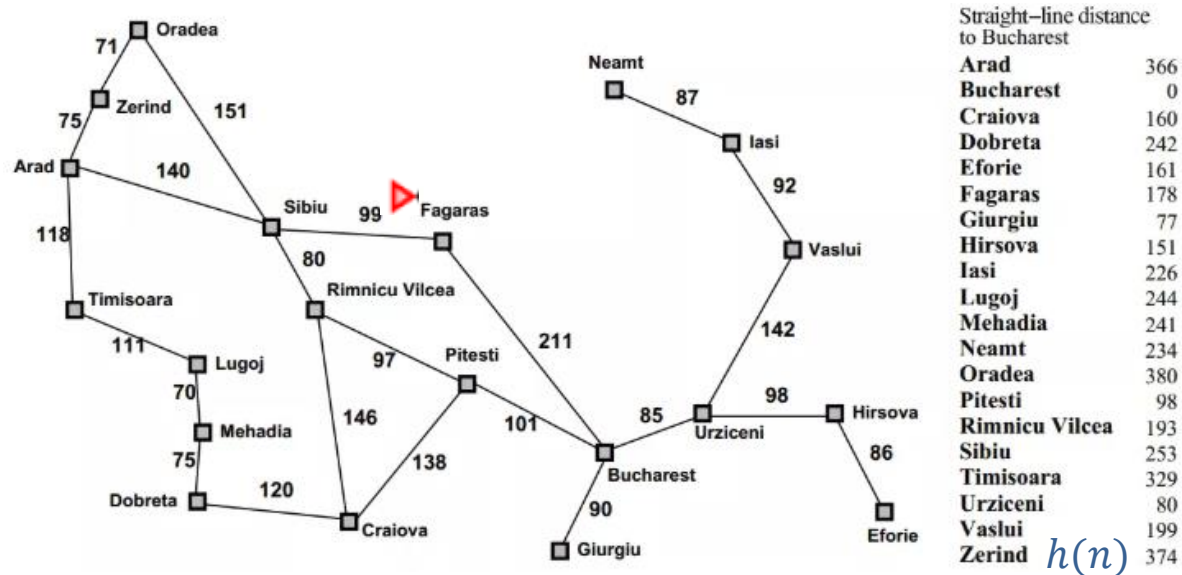
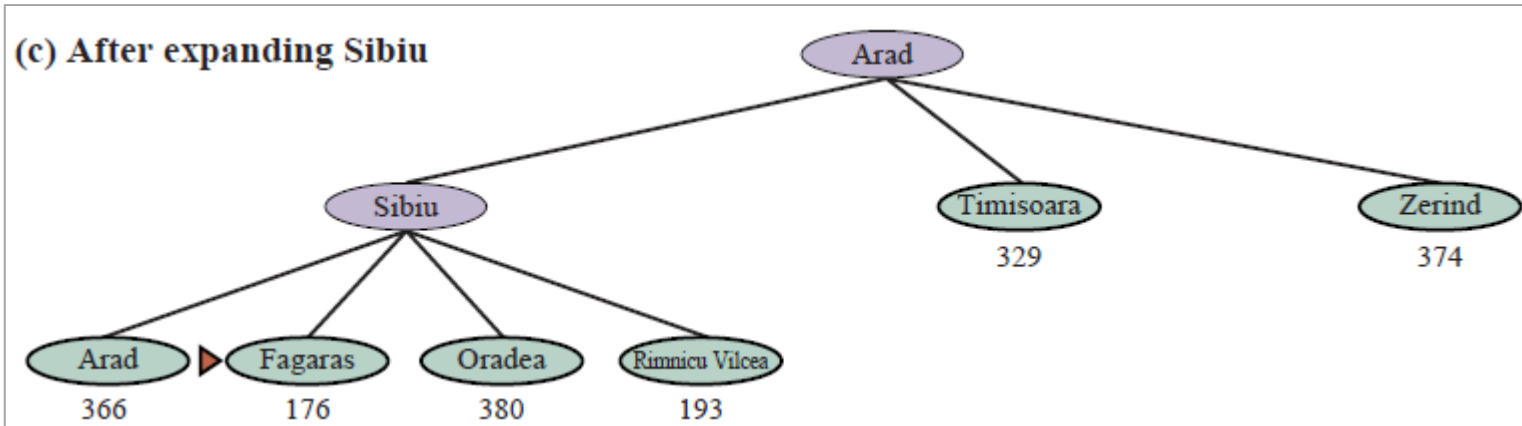
Path planning with greedy best-first



Reference: Stuart Russell & Peter Norvig, *Artificial Intelligence: A Modern Approach* (4th ed.) P103 – P104

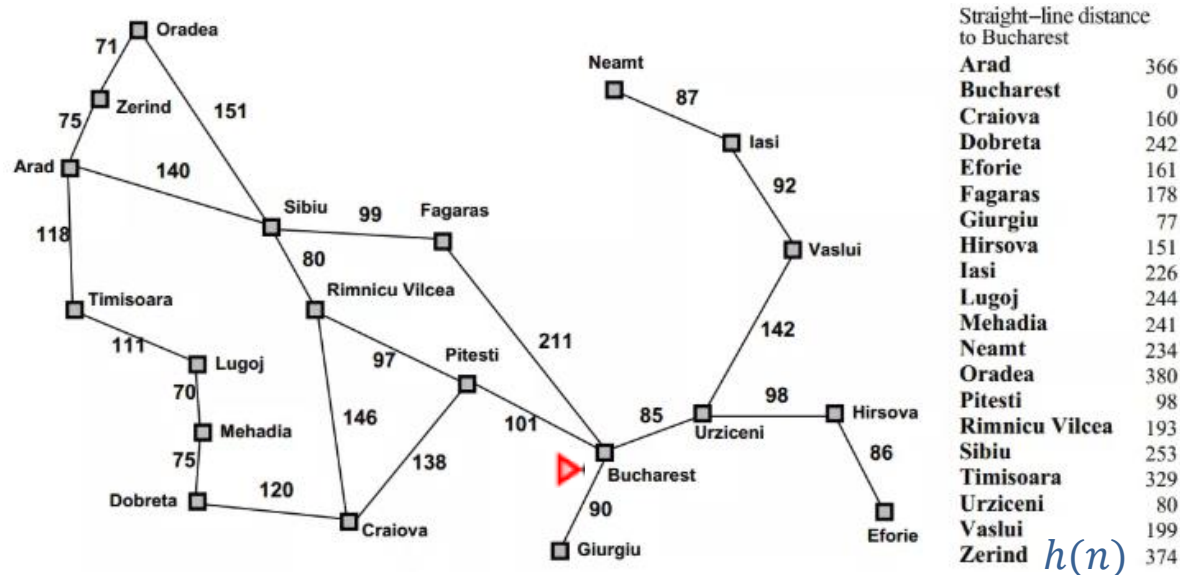
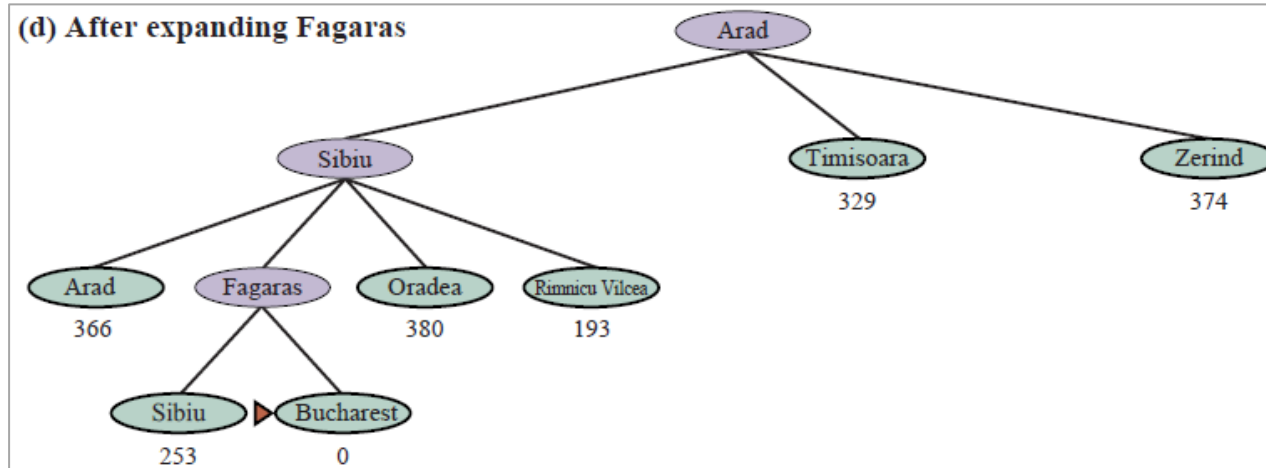
Path planning with greedy best-first

(c) After expanding Sibiu



Reference: Stuart Russell & Peter Norvig, *Artificial Intelligence: A Modern Approach* (4th ed.) P103 – P104

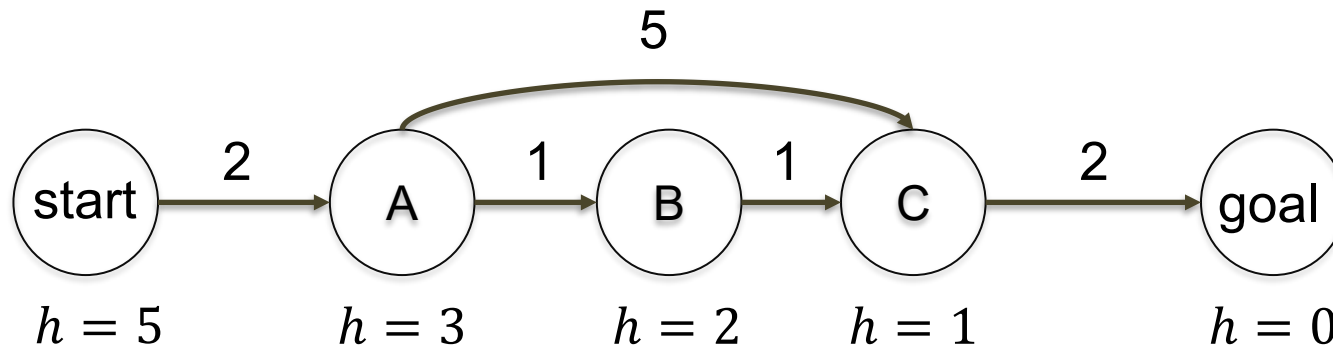
Path planning with greedy best-first



Reference: Stuart Russell & Peter Norvig, *Artificial Intelligence: A Modern Approach* (4th ed.) P103 – P104

Quick question 4

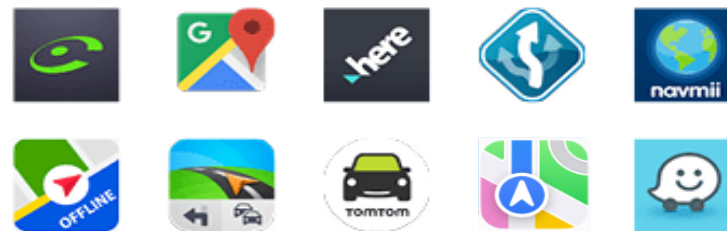
Which path does greedy best-first search return?



Each node is labeled with its heuristic estimate $h(n)$ (shown below the node), and each edge is labeled with its actual cost.

A* search: A Preview

- A* search addresses the limitation of greedy best-first search by also considering the cost so far, not just the heuristic.
- Perhaps the most well-known search algorithm in AI and pathfinding
- Widely used in navigation systems like Google Maps, Apple Maps, and other routing applications



Reference: Stuart Russell & Peter Norvig, *Artificial Intelligence: A Modern Approach* (4th ed.) P103 – P109

References

Uniform cost search:

<https://www.youtube.com/watch?v=dRMvK76xQJI>

Dijkstra's algorithm:

<https://www.youtube.com/watch?v=bZkzH5x0SKU&t=380s>

https://www.youtube.com/watch?v=EFg3u_E6eHU&t=330s

Greedy best-first search and application:

<https://www.geeksforgeeks.org/dsa/greedy-best-first-search-algorithm/>

